產品功能說明

xConsole 模型主控台

統一記憶體 AI 機的模型編組與記憶體准入控制

文件名稱	xConsole 模型主控台　主要功能說明
版本	1.2
發行日期	2026/10/22
編製單位	云碩科技股份有限公司（xCloudinfo）
聯絡窗口	李偉業　jefflee@cloudinfo.com.tw
文件密級	機密　Confidential

云碩科技股份有限公司

2026/10/22

目錄

第一章　系統定位 **3**

1.1　這套系統解決的問題 3

1.2　適用機型 3

1.3　與既有監控工具的分工 4

第二章　主要功能 **6**

2.1　記憶體准入控制 6

2.2　角色欄位 7

2.3　情境切換 7

2.4　網頁換模型 8

2.5　模型庫 9

2.6　API 串接 10

2.7　即時監控 11

2.8　記錄檢視 12

2.9　HTTPS 與憑證管理 12

2.10　服務使用報告 14

2.11　帳號與操作紀錄 15

第三章　三種記憶體架構 **16**

第四章　部署模式 **18**

4.1　單機模式 18

4.2　多機模式 18

4.3　目前的部署狀態 18

第五章　安全設計 **19**

第六章　實測數據 **21**

6.1　兩顆模型的對照 21

6.2　兩項與直覺相反的發現 21

6.3　上下文長度與顯存 22

6.4　畫面長度 22

附錄一　操作入口 **23**

附錄二　已知限制 **25**

第一章　系統定位

1.1　這套系統解決的問題

地端 AI 機器上同時跑著好幾個模型與服務：寫程式用的語言模型、抓網頁用的瀏覽器、語音辨識、語音合成、影片處理。過去要知道哪台機器開了什麼、還剩多少記憶體、能不能再多開一個，只能 SSH 進去逐台查，再靠記憶或記事本記下每個模型的參數。機器一多，這份紀錄很快就和現況脫節。

xConsole 把這件事收進一個網頁。它以「角色」而不是以「模型名稱」來排版——主腦、副腦、聽覺、說話、記憶、判斷、雙手、眼睛各占一格，空著的格子會直接顯示出來，一眼就看得出這台機器還缺什麼。

不過，這套系統真正要解決的不是「看」，而是「擋」。**統一記憶體機器一旦把記憶體吃爆，不是單一服務當掉，而是整台卡死、要走到機器前面按電源重開。** 我們的 HP Z2 Mini（Ryzen AI 395）就發生過不只一次。xConsole 在載入任何模型之前，會先試算「載下去之後最壞剩多少」，不夠就擋下來並用白話說明原因。

1.2　適用機型

xConsole 專為三個系列的統一記憶體機器設計。這三個系列的共同點是 CPU 與 GPU 共用同一塊記憶體，也正是傳統「只看顯存」的監控工具最容易判斷錯誤的地方。

表 1-1　適用機型與記憶體架構

系列	代表機型	記憶體配置	在 xConsole 中的架構代號
AMD Ryzen AI 395	HP Z2 Mini G1a	BIOS 切 96 GiB 給 GPU，系統只剩 31 GiB	unified_split
NVIDIA Jetson / 研華 MIC-743	Jetson AGX Thor、AGX Orin、Orin NX	統一記憶體單一池（Thor 為 122 GiB）	unified_single
NVIDIA DGX Spark	GB10	統一記憶體單一池（約 121 GiB）	unified_single

機群中的傳統多卡伺服器（例如 MI50 農場、P100、RTX 6000 Ada）也能納管，系統以 discrete 架構處理它們，但這不是本產品的主要對象。

一台機器也能管，十台也能管。賣一台 AI 機器給客戶時，可以把主控台直接裝在那台機器上，客戶開瀏覽器就能用；之後機器變多，再把它們串到同一個主控台，各機上的設定完全不必重做。

1.3　與既有監控工具的分工

云碩機群原本已有 amd-cluster-monitor 與 gpufleet 兩套監控儀表板，可以看到每台機器的 CPU、記憶體、磁碟與 GPU 使用率。xConsole 不重做這一層——它的硬體數據直接取自各機上已經在跑的監控代理，自己只負責監控做不到的部分：哪個角色由哪個模型擔任、載入前的記憶體試算、模型的啟停與更換、情境切換，以及各服務的記錄。

表 1-2　xConsole 與既有監控工具的分工

能力	既有監控儀表板	xConsole
硬體使用率、溫度、磁碟	有	沿用監控代理的數據
依角色排列的模型欄位	無	有
載入前的記憶體准入試算	無	有
網頁上啟動、停止、更換模型	無	有
網頁下載模型、管理存放區	無	有
IDE 與其他服務的 API 串接設定與實測	無	有
一鍵切換整組參數（情境）	無	有
檢視各服務的記錄	無	有

第二章　主要功能

2.1　記憶體准入控制

每次啟動、重啟或更換模型之前，xConsole 都會先做一次試算。它把「目前還剩多少」、「這個模型需要多少」與「這台機器的安全底線」放在一起比較，只要任何一個記憶體池在載入後會跌破底線，就擋下並說明是哪一塊不夠、差多少。

試算看的是「載下去之後最壞剩多少」，而不是「現在剩多少」。兩者的差別在於，一個模型剛載入時用量往往不高，等到真正處理長請求時才把記憶體撐開；只看當下用量，就會放行一個稍後會把機器拖垮的載入。

對已經在跑的模型做重啟時，系統會把它自己目前占用的記憶體先算回可用量。這一點看似細節，卻是實際踩過的問題：若不這樣處理，重啟一個本來就跑得好好的模型會被它自己擋住。

以 2026/10/20 在 .26 上的實測為例，兩種情境的判斷如下。

表 2-1　准入控制實測判斷

情境	系統判斷	系統給出的理由
重啟已在運作的主腦模型	放行	會先釋放它自己占用的 63.9 GiB 顯存與 16.2 GiB 系統記憶體
在同一台再多載一顆 64 GiB 的模型	擋下	系統記憶體只剩 17 GiB，載下去會剩 3.2 GiB，低於 15 GiB 底線

被擋下時，畫面會說明原因，並提供「我知道風險，仍要執行」的選項。這是刻意保留的——系統的職責是讓人在知情的狀況下做決定，而不是完全剝奪操作者的判斷。

模型需要多少記憶體，系統的估算依序採用三種來源：這台機器實際載入過該模型時量到的數值、以目前運作中模型的實測值按權重大小比例推算、最後才是保守的預設估計。模型成功載入後，系統會自動量一次真實用量並記下來，下次就不必再猜。

2.2　角色欄位

每台機器的頁面以角色排列，而不是以模型名稱排列。角色分為主腦、副腦、嵌入記憶、語音辨識、語音合成、路由判斷、產圖雙手、抓料眼睛等。每個欄位卡片會顯示目前由哪個模型擔任、量化格式、能力標籤（文字、語音、影片、網頁）、服務埠號、上下文長度、顯存與系統記憶體用量，以及實測的速度區間。

速度數字全部取自引擎自己的記錄，不是手填的規格值。系統持續讀取引擎每次處理請求後留下的紀錄，累積成「匯入提示」與「生成回答」兩組速度區間，並記下量過的最長提示長度。

服務狀態不只分「開」與「關」。啟動中、停止中、失敗都會分別標示，因為卡在啟動中的服務若被歸為「關」會讓人以為它沒在跑，若被歸為「開」又會讓人以為它已經好了，兩種都會誤導判斷。

標記為「關鍵服務」的欄位，在停止或重啟時會多一道確認，並顯示這個服務停掉之後誰會受影響。例如 .26 的主腦欄位就註明了「同事的 Copilot 後端，停掉就不能寫程式」。

2.3　情境切換

同一顆模型在同一台機器上，用途不同時需要的參數也不同。寫程式的用戶端一輪就會送出四萬個 token 以上的提示，需要很長的上下文；抓網頁清洗資料則每段只有幾千個 token，把並行數讓出來換吞吐量反而更划算。

表 2-2　預設情境

情境	上下文長度	並行數	適用用途
寫程式後端	131,072	1	GitHub Copilot 這類一次送大量程式碼脈絡的用戶端
抓料清洗	32,768	2	網頁抓取後的資料清洗，每段提示較短

切換情境時，系統會先備份目前的設定檔，再寫入新參數並重啟服務。若新設定載入失敗，系統自動還原成上一版並重新啟動，不會留下一個改壞又起不來的狀態。

2.4　網頁換模型

使用者可以直接在欄位卡片上按「換模型」，從這台機器已安裝的模型中挑選，不必 SSH 進機器、也不必知道設定檔放在哪裡；空著的欄位則按「放入」，直接放一顆模型進來。這個功能讓非管理者也能自行更換，例如寫程式的同仁可以在通用模型與程式專用模型之間自己切換。

xConsole 管兩種推論引擎。Strata 只跑 Qwen3.8-Flash-Next 這一個家族，長上下文最省記憶體；其他模型，包括 Gemma、gpt-oss、Llama，一律由 llama.cpp 執行。兩種引擎在畫面上是同一個換模型視窗，使用者不需要知道背後的差別。不論換成哪一顆，都照同一套四個步驟。

表 2-3　換模型的四個步驟

步驟	做法
1. 放哪裡	每台機器一個固定的存放區（預設 `~/models/gguf`，可改到別顆碟），一顆模型一個資料夾；分片與看圖用的 mmproj 檔放同一個資料夾
2. 怎麼放進來	在模型庫貼 Hugging Face 名稱下載，或自己把檔案複製進存放區
3. 怎麼選	換模型視窗同時列出兩種引擎的模型，標出大小、架構、最長上下文、預估記憶體與是否為中國來源
4. 怎麼切	同一個欄位、同一個埠；跨引擎時舊引擎停用、新引擎接手，IDE 的網址不必改；打不通就整組換回

記憶體需求在選的當下就算給使用者看。系統直接讀模型檔的檔頭，取出層數、每層的 KV 頭數與頭維度，算出每個 token 要多少上下文快取，再乘上使用者設定的上下文長度；滑動視窗的層只計入視窗內的 token。以一顆 30 層、每層 3 個 KV 頭、頭維度 64 的小模型為例，每個 token 需要 23,040 位元組，與手算結果一致。這個數字加上權重大小，就是套用前准入試算所用的需求。

換 Strata 家族的模型時，換的是一整份設定，而不只是兩個檔案路徑。每顆模型搭配的專家剖析檔、分片設定、快取策略都不一樣；若只替換權重路徑，引擎會因為參數對不上而在數秒內崩潰。因此系統只列出具備完整專屬設定檔的模型，套用時以該模型的設定為基礎，只保留與機器本身相關的項目。llama.cpp 的模型則由系統替欄位產生自己的設定檔與服務，固定關閉記憶體映射，避免統一記憶體機器的系統記憶體多吃一份頁快取。

套用流程依序為：備份設定、套用新模型設定、執行記憶體准入試算、重新啟動、等待服務真正回應。**判斷是否成功，看的是服務埠能否正常回應模型清單，而不是啟動指令有沒有回傳成功。** 這是實測得來的教訓：啟動指令只要被系統接受就會回報成功，模型載入失敗卻是幾秒之後才發生，若只看指令結果，系統會誤以為切換成功而不去還原。跨引擎切換時，舊引擎的開機自動啟動會一併取消，避免重新開機後兩個引擎搶同一個埠。

欄位上設定過的 API 金鑰、單次輸出上限與自訂的模型名稱都跟著欄位走，換模型或換引擎都不會改變；IDE 那邊因此不必跟著改設定。換完之後畫面會自動帶到該欄位的 API 串接頁。

為了避免有人藉這個功能指向任意檔案，系統只接受掃描清單內的模型。2026/10/20 的實測中，送出指向 `/etc/passwd` 的請求會被直接拒絕，回應為「這個組合不在本機掃到的模型清單內」。

2.5　模型庫

模型庫在頂部導覽的第三格，排在「機器」之後、「API 串接」之前，順序就是管理者實際的動線：先把模型放進來，換上欄位，再交給 IDE。第二列先是「全部機器」，一張表看完每台裝了哪些模型、哪些正在用；再來是各台機器，只看那一台。機器頁右上角也有「這台的模型庫」可以直接跳過來。頁面上方同樣有一條順序軸：確認存放區、下載或放入模型、換上欄位、到 API 串接實測，每一步的狀態都由實際資料算出，還沒完成的步驟有按鈕直接跳到負責那一步的區塊。

每台機器一個存放區，預設是 `~/models/gguf`，第一次打開模型庫時代理會自動建好。模型進來的方式有兩種：在這裡貼 Hugging Face 名稱下載，或照「手動放入」的目錄結構自己把檔案複製

進去。存放區底下一顆模型一個資料夾，同一個資料夾可以放同一顆模型的多種量化，分片要放在同一個資料夾而且要齊；看圖模型把 `mmproj` 開頭的視覺檔放在同一個資料夾。直接放在存放區根目錄的檔案不會被認到。不能連外的環境（例如醫院）就是走手動放入：模型在別台機器上時，從那台直接用 rsync 傳到這台，頁面上有可以複製的指令。

下載時先選「下載到哪台」，再貼上 Hugging Face 的模型名稱，系統會列出這個模型所有可下載的量化版本與大小，並在下載前檢查磁碟空間是否足夠。推測解碼用的輔助檔會被標示出來並排在最後，不會被誤當成最小的量化。下載由各機的代理在背景進行，一次只下載一個，可以取消；連線中斷之後再按一次會從中斷處接續。需要授權的模型可以填入 Hugging Face 權杖，權杖只在下載期間放在記憶體中，不寫入任何設定檔或紀錄。

已安裝的模型以表格列出引擎、大小、架構與最長上下文、來源，以及目前由哪個欄位使用。依公司規定，政府案不得使用中國來源的模型，模型庫會把 Qwen、DeepSeek、GLM、Kimi 等來源的模型標示出來；內部測試可以照常使用，交付政府案的機器不能安裝。沒有在用的模型旁邊有「換上」：那台只有一個可以換模型的欄位時，直接打開換模型視窗並選好這顆；有多個欄位時先問要放進哪一格。沒有在用的模型也可以從這裡刪除，正在被欄位使用的模型不能刪。

2.6　API 串接

換完模型之後，IDE（GitHub Copilot、OpenCode、Continue、Cline）與公司其他服務要知道四件事：打哪個網址、模型名稱叫什麼、上下文多長、要不要金鑰。API 串接頁把這四件事集中在一頁，而且都能在網頁上修改。

表 2-4　API 串接的設定項目

設定項目	說明
對外網址	預設是這台機器的 IP 與埠；前面有反向代理或內部網域時改成那個
模型名稱	IDE 裡要填的名稱；自己取過的名稱在換模型時會沿用
上下文長度	調大之前先做記憶體試算，不夠會擋下
單次輸出上限	要小於上下文長度；留空時由引擎決定
API 金鑰	可產生或關閉；金鑰只在產生的當下顯示一次，之後忘了就重新產生

改動模型名稱、上下文、輸出上限或金鑰需要重啟該模型。系統會先備份設定，重啟後用新設定實際打一次，確認新名稱出現在模型清單、開了金鑰之後沒帶金鑰會被擋下，才算套用成功；任何一項不通過就換回原本的設定。

頁面下方依目前設定即時產生各工具的設定範例，包括 VS Code 的 GitHub Copilot、其他 IDE 的通用欄位、OpenCode、Continue、Cline / Roo Code、Claude Code、curl 與 Python，可以直接複製。「實測一次」會在機器本機依序打模型清單、一般對話與工具呼叫，再從主控台經網路打對外網址，證明別台電腦也連得到。實測不會跟隨轉址，避免金鑰被帶到別的位址。2026/10/22 在 .27 上實測，一般對話約 1.5 秒、工具呼叫約 2.3 秒完成。

2.7　即時監控

每台機器的頁面同時顯示記憶體與硬體狀態。記憶體以堆疊長條呈現，每個運作中的模型各占一段顏色，並在長條上標出安全底線的位置，一眼就看得出距離危險還有多遠。統一記憶體單一池的機器只畫一條長條；切成兩塊的機器則分別畫出顯存與系統記憶體，因為對這類機器而言，系統記憶體那一塊才是真正會出事的地方。

硬體區塊顯示 GPU 使用率、溫度、功耗與時脈，CPU 負載與核心數，開機天數，各感測器溫度，以及磁碟用量。另有一張「監控中的埠」清單，列出這台機器上所有正在監聽的服務埠，並標示它屬於系統、模型或主控台，以及是否對外開放。

總覽頁把所有納管機器並列，顯示每台的記憶體用量、運作中的角色與目前情境；系統記憶體接近底線的機器會以醒目顏色標示。畫面每 15 秒自動更新一次。

2.8　記錄檢視

記錄頁可以選擇任一台機器的任一個服務，直接檢視它的系統記錄與引擎記錄。過去要查一個服務為什麼起不來，得先 SSH 進機器、找出記錄檔位置、再用指令過濾；現在在網頁上選好機器與服務即可看到。記錄區塊固定高度並在區塊內捲動，不會讓整個頁面無限延長。

2.9　HTTPS 與憑證管理

憑證每年要換，過去每換一次就得請工程師進主機，換錯（私鑰不成對、少了中繼憑證）整站就連不上，而且常常發生在憑證已經過期、使用者打不開的那個早上。xConsole 把這件事做成管理者在網頁上自己完成的工作，不必進主機、也不必重開服務。

畫面頂端是一條 5 步的設定順序軸：確認使用者會打的網址、取得憑證、上傳並檢查、套用、開啟 HSTS。每一步的狀態由系統實際資料算出，標示「已完成」「現在做這步」或「還不能做，要先完成第幾步」，剛接手的人一眼就知道自己卡在哪裡。

上傳區接受客戶實際會拿到的各種格式，包括 Windows 與 IIS 常用的舊式加密 PFX、分開的憑證與私鑰與中繼憑證（順序不拘）、打開是亂碼的 DER 格式 CER 檔，以及憑證廠商寄來的 P7B。系統在套用之前逐項檢查並以白話說明結果，分成「不能套用」「要注意」「可以套用」三段。例如私鑰與憑證不成對時，畫面直接寫「私鑰和憑證不是同一對」，而不是顯示 OpenSSL 的原始錯誤訊息。

表 2-5　匯入前的檢查項目

檢查項目	不通過時畫面的說明
私鑰與憑證成對	私鑰和憑證不是同一對，請確認私鑰就是當初申請這張憑證時產生的那一把
效期	憑證已在某日過期；剩 30 天內提醒盡快續約
網址涵蓋	憑證沒有涵蓋哪些網址，使用者用這些網址連線會看到憑證錯誤
憑證鏈	沒有附上中繼憑證時提醒：電腦瀏覽器會自己補，手機與其他系統可能連不上
金鑰強度	RSA 低於 2048 或 ECC 低於 256 時擋下
自簽憑證	提醒瀏覽器會出現安全警告，正式上線應改用廠商或單位 CA 簽發的憑證

套用時不重開服務，已連上的使用者不會中斷，新連線立刻使用新憑證。 系統在套用後會實際連自己一次，核對送出的憑證指紋；若不相符就自動換回原本那張。畫面另有「還原上一張」，保留最近十張可隨時換回。

還沒有憑證的單位，可以直接在頁面上產生憑證簽署請求（CSR）交給憑證廠商。私鑰在伺服器上產生並以 AES-256-GCM 加密保存，從頭到尾不離開伺服器；廠商簽回憑證之後只要上傳憑證，系統會自動對上當初那把私鑰。

HSTS 讓瀏覽器以後一律走 HTTPS。這個開關只在正式憑證套用之後才能開啟：若在暫用自簽憑證下開啟，日後憑證一出問題，使用者連「略過警告」的選項都沒有。憑證剩 30、14、7、1 天時各記一次到期提醒，剩 7 天內每一頁的頂端都會出現提醒列。

有人用 `http://` 打到主控台的埠時，系統回 308 轉到 `https://` 的同一個網址；用 308 而不用 301，是為了讓 POST 請求在轉址之後仍是 POST。

2.10　服務使用報告

主管要知道這套系統有沒有在用、設備有沒有被用到、有沒有出過問題時，答案不應該靠工程師手抄。xConsole 每 5 分鐘記錄一次各機狀態，並把每一次操作寫進稽核紀錄；服務使用報告的所有數字都從這兩份資料直接計算，在主控台上都能逐筆查到。

表 2-6　服務使用報告的六個區塊

區塊	回答的問題	資料來源
誰在用	多少次登入、多少次操作、哪些帳號	稽核紀錄
用了多少電	GPU 晶片本期耗電度數（設定電價後換算金額）	每 5 分鐘取樣的 GPU 功耗
地端處理了多少	請求數、匯入與生成的 token 量、快取命中率	各機引擎的請求記錄累計值
設備有沒有用到	各機在線比例、記憶體池平均與峰值、系統記憶體最低剩餘	每 5 分鐘取樣
連線與操作安不安全	憑證狀態、各機代理是否已建立信任、准入擋下與強制略過次數	憑證管理與稽核紀錄
系統健不健康	待處理事項清單	以上全部

估算值一律連同假設一起呈現。用電量只含 GPU 晶片自己回報的功耗，不含 CPU、記憶體與空調，報告裡會寫明；電價與雲端參考單價沒有設定時，報告只列度數與 token 量，不自行補一個數字去換算金額。

報告會揭露系統自己的問題。2026/10/22 第一次產出時，系統即列出三項待處理事項：.26 的系統記憶體曾低於安全底線、主控台仍使用暫用自簽憑證、.26 的抓料瀏覽器服務一直停在啟動中。

管理者可以在網頁上直接檢視近 7、30、90 天的數字並下載原始資料；要產出給主管看的 Word 月報，在有原始碼的電腦上執行一行指令即可，產生器會自動畫圖、組文件、轉 PDF 回填目錄頁碼並驗證內容。

2.11　帳號與操作紀錄

登入採帳號加密碼，密碼以 PBKDF2-SHA256（200,000 輪）加鹽雜湊後存檔。登入票證記在伺服器端，登出就立即作廢；管理者可以在網頁上自行變更密碼，變更後所有已登入的裝置都會被登出。同一台電腦 15 分鐘內輸入錯誤 5 次會被暫時擋下，期間連正確的密碼也不放行，讓暴力猜測無從判斷是否猜中。

所有啟動、停止、更換模型、下載與刪除模型、修改 API 設定、切換情境、更換憑證的操作，都會記下是誰、何時、從哪台電腦、做了什麼、結果如何。寫入紀錄之前一律經過同一支遮罩程式，密碼、私鑰、權杖與 PFX 內容都不會進入紀錄，憑證只記錄指紋。

第三章　三種記憶體架構

xConsole 最核心的設計，是把三種完全不同的記憶體配置分開計算。套用同一條公式，在某些機器上會盯錯數字，給出一個看起來安全、實際上危險的判斷。

表 3-1　三種記憶體架構的計算方式

架構代號	代表機型	計算方式	預設底線
unified_split	AMD Ryzen AI 395	顯存與系統記憶體分開計算，兩者都必須在底線之上	系統記憶體 15 GiB、顯存 2 GiB
unified_single	Jetson、DGX Spark	一個池子合併計算，模型需求與系統需求加總後比較	12 GiB
discrete	多卡伺服器	每張顯示卡各自計算，系統記憶體另外計算	系統記憶體 8 GiB、顯存 1 GiB

Ryzen AI 395 是最容易誤判的一種。BIOS 把 96 GiB 切給 GPU，作業系統只剩 31 GiB，因此顯存看起來很寬鬆，真正會耗盡的是那 31 GiB 的系統記憶體。過去造成整台當機的事件，都是系統記憶體耗盡而非顯存不足。15 GiB 的底線與機群既有的記憶體看門狗採用同一條標準。

Jetson 與 DGX Spark 則相反，它們的 `nvidia-smi` 在記憶體總量欄位回報的是 `[N/A]`，因為統一記憶體並不以傳統顯存的形式呈現。xConsole 對這類機器改讀作業系統的記憶體資訊，並把 GPU 與系統的需求合併計算，不會分開判斷成「兩邊各自都還夠」。

各機的底線可以在該機的欄位登錄檔中個別調整，以配合不同機型的實際狀況。

多張獨立顯示卡的伺服器（例如兩張 RTX PRO 5000 48 GB）還要多管一件事：總量夠不代表每張都夠。卡數不需要設定，代理每次量測時直接向驅動詢問，NVIDIA 由 `nvidia-smi` 每張卡回報一列，AMD 由作業系統的顯示卡目錄列出；客戶的機器只有一張卡時，下面這些選項都不會出現，畫面與以前相同。

機器頁的顯存先畫所有卡的合計，再每張卡各畫一條，並標出綁在那張卡上的欄位。換模型時多一組「用哪幾張卡」，系統會先依模型的權重與上下文算出需要多少顯存，預設帶入建議：一張卡放得下就只用剩餘最多的那一張，少了跨卡傳輸，其餘的卡還能再跑第二顆模型；一張放不下才用最少的張數一起承擔。管理者可以改勾，不勾任何一張代表全部卡都用。

准入試算也改成逐張計算。模型依卡的容量比例分攤到它使用的每一張卡，任何一張載入後會低於底線就擋下，理由會點名是哪一張卡。改綁到別的卡時，舊模型釋放的顯存只會還給它原本使用的卡，不會誤算到新的卡上。啟動時以環境變數限定引擎只看得到指定的卡；NVIDIA 的卡號與驅動的編號順序一致，AMD 則經作業系統的拓撲資訊換算，換算不出來就拒絕啟動，不會冒險綁到別張卡。

模型的上下文快取（KV）占用多少記憶體，也因模型而異，差距可達數個數量級。目前使用的 Qwen3.8-Flash-Next 採用線性混合注意力架構，每個 token 只需 1,056 位元組，128K 上下文全開也只有 0.13 GiB；傳統全注意力模型同樣長度則可能占用數十 GiB。xConsole 允許每個欄位指定自己模型的每 token 用量，不會一體套用同一個數字。

第四章　部署模式

4.1　單機模式

單機模式把節點代理與主控台裝在同一台機器上，這台機器只管理自己。這是賣一台 AI 機器給客戶時的標準做法：客戶拿到機器、開瀏覽器就能看到自己機器上的模型狀態並進行操作，不需要另外準備一台管理用的電腦。部署只需要一行指令。

4.2　多機模式

多機模式由一台機器擔任主控台，其餘機器各自執行節點代理。主控台彙整各機狀態，以群組方式呈現在同一個畫面上。由單機模式擴充為多機模式時，各機上的代理與欄位登錄完全不必重做，只需把新機器加入主控台的納管清單。

4.3　目前的部署狀態

表 4-1　目前納管的機器（2026/10/22）

機器	群組	記憶體架構	角色
.27（192.168.11.27）	AMD AI 395 Strix Halo	unified_split	主控台所在機器，同時為抓料清洗引擎
.26（192.168.11.26）	AMD AI 395 Strix Halo	unified_split	寫程式後端（同事 Copilot 使用）
.20（192.168.11.20）	Jetson / MIC-743	unified_single	醫美廣告影片產線

主控台網址為 `https://192.168.11.27:9400`。

第五章　安全設計

xConsole 能夠啟動與停止其他服務，因此在設計上把它當成一套需要防護的系統，而不是內部小工具。

表 5-1　安全設計

面向	做法
連線加密	只提供 HTTPS（TLS 1.2 / 1.3）；沒有正式憑證時自動產生暫用自簽，不退回 HTTP；http 打到同埠會被轉走
登入	密碼以 PBKDF2-SHA256（200,000 輪）加鹽雜湊，不存明文；同一來源 15 分鐘錯 5 次即擋
登入票證	記在伺服器端、資料庫只存雜湊；登出、換密碼即作廢；只接受 JSON 格式的請求，跨站表單送不進來
操作紀錄	所有操作寫入稽核紀錄，寫入前經過同一支遮罩程式，密碼、私鑰、權杖不落地，憑證只記指紋
待配對憑證的私鑰	以 AES-256-GCM 加密後存放，主金鑰檔權限 0600
機器間的權杖	主控台端以 AES-256-GCM 加密存放；只走 Authorization 標頭、不進網址；瀏覽器看不到任何代理權杖
模型的 API 金鑰	只在產生時顯示一次，不寫入紀錄；交給引擎時用檔案，不放在程序的指令列上
主控台與各機的連線	同樣走 HTTPS，以部署時經 SSH 取回的各機憑證驗證對方，不符就拒絕連線
網頁防護	內容安全政策不允許任何行內腳本與樣式，並禁止被嵌入框架、禁止 MIME 類型猜測
換模型與下載的路徑限制	換模型與刪除只接受掃描清單內的模型，下載只接受 Hugging Face 檔案清單內的檔名

面向	做法
資源上限	代理與主控台都設有記憶體與 CPU 用量上限，避免它們本身成為壓垮機器的原因
破壞性操作	停止關鍵服務、強制略過准入試算、停止全部服務、刪除模型，皆需在畫面上二次確認
安全開發流程	每次修改都經 GitHub Actions 執行測試與六道安全掃描，全綠才能合併；安全計畫、威脅模型與交付證據包見 手冊/資安證據/

第六章　實測數據

以下數字皆為 2026/10/20 在實機上量測所得，不是規格值或估算值。

6.1　兩顆模型的對照

我們在 .27 上安裝兩顆模型，以同一份 68,622 token 的長提示、在無快取的狀態下量測。

表 6-1　兩顆模型的實測對照

項目	通用模型 UD-IQ4_XS	程式專用模型 Coder IQ1_M
權重大小	87 GiB	54 GiB
匯入提示速度	1,011.6 tok/s	1,017.5 tok/s
生成回答速度	46.0 tok/s	45.4 tok/s
該次請求從硬碟讀取	70,563 MB	9.1 MB
載入後剩餘顯存	33,094 MiB	66,364 MiB
中文能力	正常	官方說明中文會出錯或重複迴圈

程式專用模型的速度與通用模型幾乎相同，但顯存用量少了一半。它的價值不在於更快，而在於騰出空間，讓同一台機器可以同時再跑另一個模型。由於它的中文能力明顯較弱，不適合用於中文資料清洗。

6.2　兩項與直覺相反的發現

第一，長提示的速度瓶頸在運算而不在硬碟。我們原本觀察到一次請求會從硬碟讀取 70 GB 資料，判斷硬碟是瓶頸；但程式專用模型把讀取量降到 9.1 MB 之後，匯入速度幾乎沒有變化。硬碟讀取確實發生，只是被其他運算重疊掉了，並未拖慢整體速度。

第二，提示快取的效益遠大於更換模型。同一份 68,622 token 的提示，在快取命中時只需 61 毫秒就讀完（其中 68,615 個 token 直接重用），整筆請求由 68.2 秒縮短為 24.7 秒。寫程式的用戶端每一輪送出的都是「同一份脈絡再加長一點」，因此快取命中率比模型本身的選擇更值得關注。

6.3　上下文長度與顯存

把上下文長度由 32,768 提高到 131,072，載入後的剩餘顯存由 33,026 至 33,416 MiB（同一設定連續量兩次）變為 33,094 MiB，幾乎沒有變化。這驗證了目前模型的上下文快取極小，可以放心給寫程式的用戶端使用長上下文。

6.4　畫面長度

依云碩網頁設計規範，內頁長度不應超過螢幕高度的 2.5 倍，手機不應超過 3.0 倍，以避免使用者需要一直往下捲動。

表 6-2　頁面長度實測（2026/10/21）

螢幕寬度	機器頁面長度	規範上限
1440（桌上型）	1.80 倍	2.5 倍
1280（筆記型）	2.03 倍	2.5 倍
375（手機）	2.71 倍	3.0 倍

附錄一　操作入口

表 A-1　操作入口

項目	內容
主控台網址	https://192.168.11.27:9400
連線條件	須位於公司內網，於外部使用時須先連線 VPN
瀏覽器警告	目前使用暫用自簽憑證，第一次開啟會出現安全警告，確認後即可進入
帳號	admin；密碼交接時另行提供，可於「設定 → 帳號與安全」自行變更
原始碼位置	Desktop/08.開發環境/claude/xConsole/code

常用的維運指令如下。

表 A-2　常用維運指令

用途	指令
忘記密碼時重設	在 .27 執行 `cd ~/xconsole && python3 xconsole_server.py --set-credentials`（會互動詢問帳號與密碼）
產出服務使用報告	`/usr/bin/python3 scripts/report/build_usage_report.py --days 30 --password-file ~/.xconsole/report.pw`
單機模式部署	`bash deploy_standalone.sh <機器別名>`
加入一台機器	`bash deploy_agent.sh <機器別名> --seed`，再執行 `bash deploy_console.sh 27 <機器別名清單>`
自我檢查	八支測試與一支真機冒煙測試，清單與跑法見原始碼 README「六、自我檢查」；每次修改都在 GitHub Actions 自動重跑
指定模型存放區	在該機 `~/.xconsole/registry.json` 的 `machine` 加 `"gguf_root": "<路徑>"`

附錄二　已知限制

以下事項在本版本尚未處理，列出以免使用者誤以為已具備。

表 B-1　已知限制

項目	目前狀況	影響
模型的 API 走 http	推論引擎本身不提供 HTTPS，金鑰與提示內容在內網以明文傳輸	預計 2026/12/31 前由節點代理加一層 HTTPS
管理者登入沒有兩步驟驗證	以登入節流、內網限定與換密碼作廢補強	交付政府案前依客戶要求補上
工具呼叫看模型	llama.cpp 能不能呼叫工具取決於模型自己的對話範本，小模型常常不會	IDE 的代理模式要先用「實測一次」確認
模型下載來源	網頁下載只支援 Hugging Face；Strata 家族仍由 Strata 的安裝程式安裝	其他來源請自行複製進存放區
DGX Spark 實機納管	准入公式已通過自我檢查，但尚未接上實機驗證	DGX Spark 上的實際行為尚待確認
單一 GPU 的顯存歸屬	單一 GPU 機器無法逐一區分各程序的顯存，改依各欄位的預估需求比例分攤	同機多個模型並存時，各欄位顯示的顯存為估計值
系統記憶體量測	以程序常駐記憶體量測，含共用頁面，數值會隨時間浮動	單次讀數可能與長期平均有落差
第三方滲透測試	已做開發團隊自測與自動化弱點掃描（見證據包），尚未做第三方測試	交付中級以上政府案前依合約安排